

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконала: студент(-ка) групи AI-241
Давидович Катерина
Дмитрівна

Перевірив: ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	134	5
Час, с	10,67	5,00
Токенів/с	12.55	1

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	0 / 5
eval_count	1310,4	184,6
Час, с	84,0016	7,047

Типова помилка інструктивної моделі: відповідь «50 грн», отримана через неправильне складання рівняння.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: У міркувальній моделі (qwen3:4b) лівова частка токенів (134 проти 5 у llama3.2) пішла на прихований або явний ланцюжок думок (Chain-of-Thought, блоки `<think>...</think>`). Модель аналізує запит, валідує факти й формулює фінальну відповідь через покрокове внутрішнє міркування, навіть коли завдання елементарне.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Багатокрокова логіка, математичні розрахунки, алгоритмічне програмування, головоломки, аналіз суперечливих умов (що підтверджує результат 5/5 проти 0/5 у Таблиці 2).

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивна модель: навчається переважно за схемою SFT (Supervised Fine-Tuning) на парах «запит – пряма правильна відповідь» та вирівнюється через RLHF/DPO за загальними уподобаннями людини. Міркувальна модель: навчається генерувати довгі траєкторії міркувань (CoT) за допомогою цільового навчання з підкріпленням (наприклад, RL з верифікованими винагородами за точність результату, як у підходах DeepSeek-R1 чи OpenAI o-серії), де заохочуються самоперевірка, пошук альтернативних шляхів і рефлексія перед фінальним висновком.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні. Сама по собі надлишкова довжина генерації (test-time compute) дає моделі обчислювальний простір для пошуку правильного рішення на складних задачах, проте на простих фактах це лише надлишкові обчислення (overthinking). Крім того, модель може згенерувати довгий ланцюжок міркувань і все одно дійти хибного висновку.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Обчислення в контексті довжини генерації: при формуванні лише 5 токенів (llama3.2) виміряна середня швидкість (1 ток/с) виглядає аномально низькою через те, що накладні витрати (час завантаження моделі в пам'ять / TTFT – час до першого токена) поділилися на мізерну кількість токенів. Довга послідовність: qwen3 згенерувала 134 токени, тому етап власне декодування розгорнувся на повну швидкість (12.55 ток/с), розмивши стартову затримку. Квантування та оптимізація рантайму: різниця в типах квантування ваг (наприклад, Q4 проти Q8 чи FP16), розмірі словника токенизатора та апаратній оптимізації конкретної архітектури в русії інференсу (Ollama/llama.cpp).

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 26,8 раза за токенами (134 проти 5) та у 2,1 раза за часом (10,67 с проти 5,00 с). На задачі на міркування ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді (5 з 5 правильних у qwen3 проти 0 з 5 у llama3.2). Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 672,0 с проти 56,4 с (понад 11 хвилин очікування проти менш ніж 1 хвилини), що робить міркувальну модель непридатною для швидкого інтерактивного налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Варіант 9 – Поясни різницю між функцією і методом.

Таблиця 3 – Вплив system повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1567	українська з русизмами і латинськими вставками	12	Ні
llama3.2:3b	українською, 2 речення	182	українська з англійською вставкою («step – step»)	1	Частково
qwen3:4b	лаконічний асистент	82	українська	2	Так

qwen3:4b	українською, 2 речення	150	українська	2	Так
----------	---------------------------	-----	------------	---	-----

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	517	567
llama3.2:3b	українською, 2 речення	54	53	107
qwen3:4b	лаконічний асистент	55	2648	2703
qwen3:4b	українською, 2 речення	64	1565	1629

Відповісти на питання (по 1–2 речення):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Обидві формально вклалися в ліміт: llama3.2:3b написала одне речення, qwen3:4b два. Але повністю інструкцію виконала лише qwen3:4b, бо в llama до речення потрапила англійська вставка «step – step», а зміст вийшов неточним.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Так. У llama3.2:3b навіть з інструкцією «лише українською» з'явилась англійська вставка «step – step». У першому запуску, де мову не задавали, відповідь змішувала українську з російськими словами й латиницею посеред слів. qwen3:4b відповідала українською обидва рази, лише з одним русизмом «кусок» замість «шматок».

3. Як змінився prompt_tokens при зміні system повідомлення і чому?

Число зросло: у llama з 50 до 54, у qwen з 55 до 64. Нове system-повідомлення довше, а prompt_tokens рахує весь вхід разом зі службовими

токенами шаблону чату. У qwen приріст більший, бо в неї інший токенизатор і вона ділить кириличний текст на більше tokenів.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

qwen3:4b – міркувальна модель: перед відповіддю вона генерує прихований ланцюжок роздумів (блок thinking), і ці токени теж входять у completion_tokens. Тому 82 видимі символи коштували 2648 tokenів, тоді як llama одразу пише відповідь і витрачає токени тільки на видимий текст.

Крок 7. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць і поставити п'ять питань:

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system повідомленнями. Чому моделі по різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Це залежить від того, на яких даних і як ретельно модель донавчали виконувати інструкції, а також від частки української в її навчальних даних. Міркувальна модель спершу «проговорює» завдання і обмеження, тож рідше їх губить. Інструктивна модель на 3B параметрів пише відповідь одразу, і на неї сильніше тиснуть звичні шаблони, як-от розлогі пояснення з прикладами.

2. Чим system повідомлення відрізняється від user повідомлення з точки зору моделі – чи справді воно має більший пріоритет?

Технічно обидва повідомлення перетворюються на один потік tokenів, лише зі службовими мітками ролей. Вищий пріоритет system – не жорстке правило, а поведінка, якої модель навчилася під час донавчання: вона довіряє system більше, бо так її тренували. Тому пріоритет цей статистичний, і малі моделі тримаються його слабше.

3. Чому міркувальна модель витрачає більше вихідних tokenів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Перш ніж написати фінальну відповідь, вона генерує внутрішні роздуми, які користувач здебільшого не бачить, а оплачуються і рахуються вони як звичайні

вихідні токени. У моєму досліді на кілька речень відповіді пішло 1,5–2,6 тисячі tokenів, у 30–50 разів більше, ніж у Llama на короткій відповіді. Через це міркувальна модель повільніша і дорожча, зате точніша.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, – яку вона виконає?

Здебільшого добре навчена модель виконає system, особливо якщо суперечність стосується формату чи мови. Але гарантії немає: малі моделі часто слухаються останньої чи конкретнішої інструкції, тобто user. Саме на цьому будуються атаки prompt injection, тому критичні обмеження не варто тримати лише в system.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

У малій моделі мало параметрів, тож знання менш поширених мов вона тримає гірше, а української в навчальних даних значно менше, ніж англійської чи російської. Коли модель «не впевнена» у слові, вона підставляє близьке з іншої мови; звідси «Определяемо» і «вступами» у Llama. Великі моделі мають більше ресурсу, щоб розрізняти близькі мови й не змішувати їх.

Висновок за завданням 2

System-повідомлення – найдешевший спосіб керувати моделлю: одним рядком я скоротила відповідь Llama з 1567 до 182 символів, а витрати tokenів з 567 до 107. Проте надійність цього керування залежить від моделі. qwen3:4b виконала обидві інструкції повністю, а Llama3.2:3b проігнорувала «лаконічність» і навіть із прямою вказівкою вставила англійські слова. Отже, самої інструкції не завжди досить, щоб отримати потрібний формат, особливо від малої моделі. Якщо модель ігнорує інструкцію, варто сформулювати її конкретніше й вимірюваніше (наприклад, «не більше 300 символів» замість «лаконічно»), додати приклад потрібної відповіді, знизити temperature, взяти сильнішу модель, а формат перевіряти програмно й за потреби повторювати запит.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Варіант 9 – Поясни різницю між функцією і методом. Коротко.

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.8-flash
Час відповіді, с	28,98	12,09
completion_tokens	598	893
Довжина відповіді, символів	1657	2581
Мова відповіді	Українська, але сильно спотворена	українська
Фактичних помилок у відповіді	4	0

Крок 5. Відповісти на питання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Лише два рядки в словнику PROVIDERS у providers.py: base_url і model (ключ читається з .env). У compare.py не змінилося нічого: виклик create() однаковий для обох провайдерів, змінюється тільки конфігурація.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Повніша й точніша відповідь хмарної моделі. Gemini зрозуміла, що питання про програмування, і пояснила головне: метод – це функція, оголошена всередині класу, яка має доступ до об'єкта через self/this. Відповідь супроводжують приклади коду на Python (add(5, 3) проти my_dog.bark()), порівняльна таблиця й аналогія. llama3.2:3b натомість пояснила «функцію» як математичне відображення

між множинами, а «метод» як спосіб розв'язання задачі, тобто відповіла взагалі не на те питання.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Так, і тих, і тих багато. Фактичні помилки: питання про програмування сприйняте як математичне; «метод Ньютона використовується для визначення функцій» – насправді він шукає корені рівнянь; «метод Гаусса використовується для оптимізації функцій» – насправді він розв'язує системи лінійних рівнянь; беззмістовні фрази на кшталт «функція є еліпсічним оспібітом». Проблеми з українською мовою: гібриди «Relationsній», «applicірується», «systematicкий», неіснуючі слова «стілярностініх», «ніжачиги», русизм «связі».

4. Хмарна модель відповіла швидше за локальну – за рахунок чого, якщо запит летить через інтернет?

Мережа додає лише десятки чи сотні мілісекунд, а основний час іде на генерацію токенів. Gemini працює на серверних прискорювачах Google (TPU) з величезною пропускнуою здатністю пам'яті. За 12 с вона згенерувала близько 1930 токенів (разом із прихованими), тобто приблизно 160 токенів/с. Ноутбук видав 598 токенів за 29 с, приблизно 20 токенів/с.

Крок 6. Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема Ollama?

OpenAI першою запустила масовий API для чат-моделей, і під нього написали тисячі програм, бібліотек та інструментів. Іншим провайдерам вигідніше підтримати вже знайомий формат, ніж змушувати розробників переписувати код, бо так до них легше перейти. Для Ollama це означає, що локальна модель одразу працює з будь-яким кодом, написаним під OpenAI.

2. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Розмір – один із найважливіших чинників: у 3В-моделі просто не вистачає «місця», щоб надійно зберігати знання й мови, і мій дослід це показав. Але важать і якість навчальних даних (зокрема частка української), донавчання на виконання інструкцій і наявність міркування. Тому сучасні малі моделі часто кращі за старі великі, хоча розрив з хмарними моделями все одно великий. Примітка: завдання згадує хмарну модель на 70 млрд параметрів (мається на увазі Groq), але Google не розкриває розмір Gemini, тож у своєму питанні чат-боту краще написати «з хмарною моделлю невідомого, але значно більшого розміру».

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Мережева затримка становить частки секунди, а генерація триває секунди, тому все вирішує швидкість обчислень. Дата-центри мають спеціалізовані прискорювачі з дуже швидкою пам'яттю й оптимізоване обслуговування запитів, а ноутбук ділить ресурси з іншими програмами. Швидкість генерації в хмарі може бути в рази чи десятки разів вищою, що з запасом перекриває мережеву затримку.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Усе, що ви надсилаєте, потрапляє на сервер провайдера: там воно може зберігатися в логах, його можуть переглядати співробітники, а на безкоштовних тарифах дані іноді використовують для навчання моделей. Комерційні проєкти укладають угоди про обробку даних, обирають платні тарифи без навчання на даних, видаляють чи маскують персональні дані перед відправкою, а найчутливіше обробляють локальними моделями на власних серверах.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit – це обмеження кількості запитів або токенів за хвилину чи добу; при перевищенні сервер повертає помилку 429. Розробники повторюють запит із

поступово зростаючою паузою (exponential backoff), кешують відповіді, об'єднують кілька запитів в один, перемикаються на резервного провайдера або переходять на платний тариф.

Висновок за завданням 3

Локальну модель я обрала б там, де важливіші приватність і незалежність, ніж якість: для роботи з конфіденційними даними (медичними, фінансовими, внутрішніми документами компанії), без інтернету, для простих масових задач на кшталт класифікації коротких текстів, а також для навчання й експериментів без лімітів і витрат. Хмарну модель я обрала б для задач, де потрібні точність і добра мова, як-от пояснення, генерація тексту чи код, коли дані не секретні. Мій дослід це наочно показав: на те саме питання llama3.2:3b дала неправильну відповідь поламаною мовою, а Gemini вдвічі швидше дала точну, структуровану відповідь з прикладами. Найпрактичніше поєднувати обидва підходи. Завдяки єдиному формату API перемикання між ними – це зміна двох рядків конфігурації, тож можна направляти чутливі запити на локальну модель, а решту в хмару.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Варіант 9:

Столиця Туреччини?

Напиши інструкцію з 10 пунктів, як оптимізувати повільний застосунок.

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	0,144	2,311	55,0
локальна	llama3.2:3b	середній	0,118	5,745	66,2
локальна	llama3.2:3b	довгий	0,226	29,533	63,7
хмарна	gemini-3.5-flash-lite	короткий	60,099	60,301	2,0

хмарна	gemini-3.5-flash-lite	середній	31,220	31,933	14,7
хмарна	gemini-3.5-flash-lite	довгий	25,897	34,725	87,8

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2,838	0,074
Всього, с	3,516	0,589

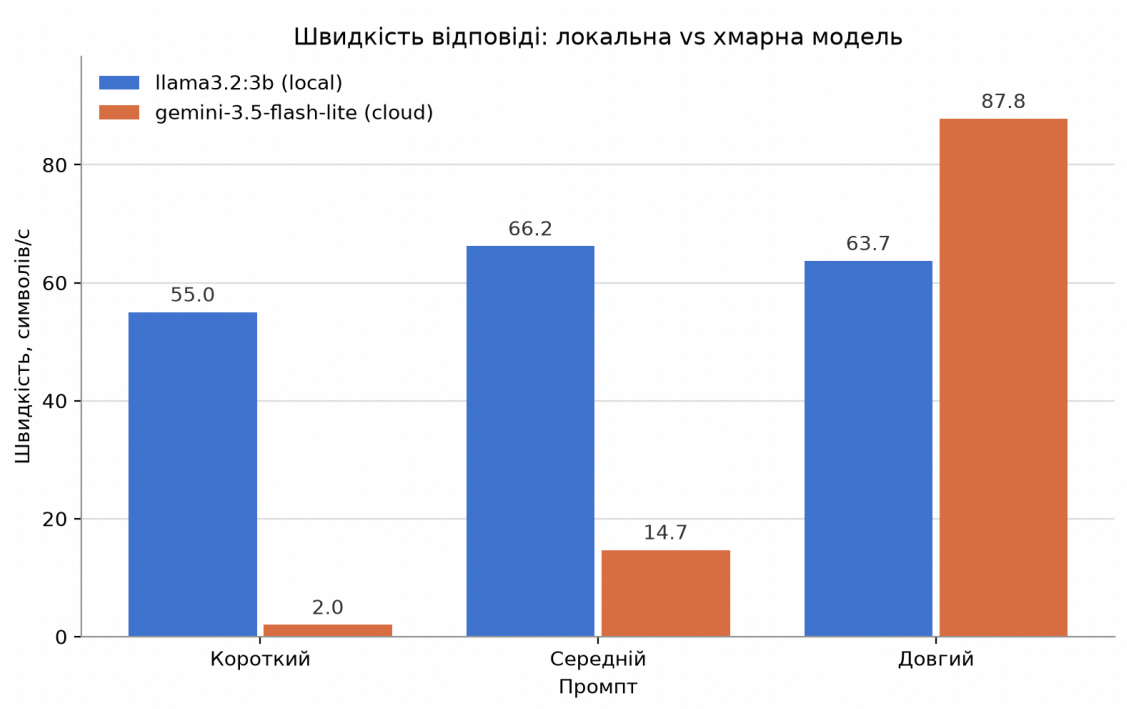


Рисунок 1 – стовпчикова діаграма

Крок 6. Відповіді на питання

1. У якої моделі кращий TTFT і чому саме в неї?

У локальної llama3.2:3b: 0,12–0,23 с проти 26–60 с у Gemini. Запит не йде мережею і не стоїть у черзі серед тисяч інших користувачів, а модель уже завантажена в пам'ять завдяки прогріву. У хмарі TTFT складається з дороги по мережі, черги на перевантаженому сервері (той самий «high demand», що давав помилку 503) і, ймовірно, прихованого міркування моделі.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні – поясніть.

Не збігається: на довгому промпті за TTFT виграє локальна модель, а за символами/с хмарна (87,8 проти 63,7). Показник символів/с рахується з урахуванням TTFT, тож чиста швидкість генерації в хмарі ще вища. Після першого токена Gemini видала близько 3000 символів приблизно за 9 с, тобто близько 345 символів/с, тоді як Llama генерує стабільно 60–65 символів/с.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

На короткому промпті локальна модель відповіла приблизно в 26 разів швидше (2,3 проти 60,3 с), бо вся відповідь хмари – це фактично очікування першого токена. На довгому загальний час майже зрівнявся (29,5 проти 34,7 с), а за символами/с хмара вже випередила локальну модель. Що довша відповідь, то більше швидка генерація хмари компенсує її довгий старт.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Дуже сильно, особливо в хмарі: TTFT на короткому промпті коливався від 36,8 до 100,3 с, майже втричі. У локальній моделі TTFT стабільніший (0,08–0,47 с), а загальний час на короткому промпті коливався від 1,3 до 5,3 с, бо модель щоразу генерувала відповідь різної довжини. Одного запуску недостатньо: він міг випадково дати і 37, і 100 с, а медіана трьох відсікає такі викиди.

5. Чому холодний старт повільніший за теплий?

При холодному старті Ollama спершу завантажує ваги моделі (близько 2 ГБ) з диска в пам'ять і готує їх до обчислень, і тільки потім обробляє запит. Тому TTFT зріс з 0,074 до 2,838 с, а сама генерація після першого токена тривала однаково, близько 0,5 с. При теплому старті модель уже в пам'яті, і цей крок пропускається.

Крок 7. Відповіді на питання до чат-бота (2–3 речення)

1. Що фізично відбувається під час TTFT і чому його вимірюють окремо від швидкості генерації?

За час TTFT запит доходить до сервера, чекає в черзі, токенізується, а потім модель обробляє весь вхідний текст (цей етап називається *prefill*) і обчислює перший токен. Далі кожен наступний токен генерується окремим кроком, і від швидкості цих кроків залежить швидкість генерації. Ці показники вимірюють окремо, бо вони залежать від різних речей і по-різному впливають на досвід користувача: TTFT визначає, скільки людина дивиться на порожній екран, а швидкість генерації – як довго «друкується» відповідь.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації?

До TTFT хмари додаються мережа, авторизація й черга серед запитів інших користувачів, чого немає в локальній моделі. Зате сама генерація відбувається на потужних серверних прискорювачах зі швидкою пам'яттю, які видають токени в рази швидше за ноутбук. У моєму досліді хмара генерувала приблизно в 5 разів швидше, але стартувала в 100–400 разів повільніше.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт – це запит до моделі, яка ще не завантажена в пам'ять, тому спочатку треба прочитати її ваги з диска й підготувати їх до обчислень. Ollama тримає модель у пам'яті кілька хвилин після використання, тож наступні запити обходяться без цього кроку. У мене холодний старт додав приблизно 2,8 с до TTFT, а повторні «холодні» заміри були швидшими, бо macOS кешувала файл моделі в оперативній пам'яті.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

На кожен запуск впливають випадкові чинники: навантаження на сервер, інші програми на комп'ютері, різна довжина згенерованої відповіді. Одне значення може виявитися викидом, а медіана, на відміну від середнього, стійка до викидів. Наприклад, з хмарних TTFT 36,8, 60,1 і 100,3 с медіана дає 60,1 с, і випадкові 100 с не завищують результат, як це зробило б середнє (65,7 с).

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Вхідний текст модель бачить одразу весь, тому може обробити всі його токени паралельно за один прохід. Відповідь генерується послідовно: кожен новий токен залежить від попередніх, тож на кожен потрібен окремий прохід моделі. До того ж при генерації вузьким місцем стає швидкість читання ваг з пам'яті, яке доводиться повторювати для кожного токена.

Висновок

Для інтерактивного чат-бота найважливіший TTFT, бо саме його користувач відчуває як «гальмування». Відповідь, що почала з'являтися за 0,2 с, сприймається як миттєва, навіть якщо друкується ще 20 секунд, а 60 секунд порожнього екрана, як у Gemini в моєму замірі, для чату неприйнятні. Для фонові обробки великих обсягів тексту (переклад документів, класифікація тисяч листів, генерація звітів) TTFT майже не важливий, бо ніхто не дивиться на екран. Там вирішують швидкість генерації і пропускну здатність, тож хмарна модель, яка в моєму досліді генерувала приблизно в 5 разів швидше, для таких задач вигідніша, попри довгий старт. Також мої заміри показали, що хмара нестабільна: той самий запит тривав то 37, то 100 секунд, тож для інтерактивних сервісів на хмарних API потрібні повтори при помилках і запасний провайдер.

Завдання 5. Оцінювання якості моделей за рубрикою

Три моделі – llama3.2:3b, qwen3:4b та хмарна gemini-3.5-flash-lite – отримали однаковий набір із шести завдань варіанта 9 (тема «Астрономія»). Кожне завдання прогнано двічі при temperature = 0,7, відповіді збережено у quality.json. Набір завдань і текст для стиснення зафіксовано до першого запуску, еталонні відповіді перевірено за авторитетними джерелами.

Набір завдань варіанта 9 і еталонні відповіді

№	Промпт	Правильна відповідь / критерій
1. Факт	У якому році відбувся перший політ людини в космос? Відповідай лише роком.	1961 (12 квітня 1961 р., Ю. Гагарін, корабель «Восток-1»)
2. Формат	Дай JSON з полями event, year, person про перший політ людини в космос. Без пояснень, без markdown, лише JSON.	Відповідь проходить json.loads() без правок
3. Стиснення	Стисни наступний текст до двох речень, збережи всі числа: <текст нижче>	Два речення, усі 9 чисел: 2015; 1,3 млрд; 36; 29; 62; 3; 4 км; 1/10 000; 1916
4. Код	Напиши функцію Python light_travel_time(distance_km), яка повертає час у секундах, за який світло долає відстань distance_km. Лише код.	distance_km / 299 792,458. Перевірка: 299 792,458 км → 1 с; 384 400 км (до Місяця) → 1,282 с; 149,6 млн км (до Сонця) → 499 с
5. Логіка	Космічний зонд летить зі швидкістю 60000 км/год. Відстань до цілі 480 млн км. Скільки повних діб триватиме політ? Покажи хід розв'язання.	$480\,000\,000 / 60\,000 = 8000$ год; $8000 / 24 = 333,3 \rightarrow 333$ повні доби
6. Українська	Чим зоря відрізняється від планети? Поясни українською, два абзаци.	Зоря світить власним світлом завдяки термоядерному синтезу; планета світить відбитим світлом і обертається навколо зорі

Текст для завдання 3: «Гравітаційні хвилі – це своєрідні “хвилі” у просторі-часі, які виникають під час дуже потужних космічних подій. Уперше їх безпосередньо зафіксували у 2015 році, коли зіткнулися дві чорні діри на відстані близько 1,3 млрд світлових років від Землі. Маса цих чорних дір становила приблизно 36 і 29 мас Сонця, а після злиття утворилася чорна діра масою близько 62 Сонць. Решта 3 сонячних мас перетворилася на енергію гравітаційних хвиль за частки секунди. Хвиля змінила довжину 4-кілометрових плечей детектора LIGO

приблизно на величину, меншу за 1/10 000 діаметра протона. Цей сигнал став першим прямим доказом існування гравітаційних хвиль, передбачених Альбертом Ейнштейном ще 1916 року.»

Джерела: текст – LIGO, «Observation of Gravitational Waves from a Binary Black Hole Merger» (<https://ligo.org/science-summaries/gw150914/>); еталон до завдання 1 – NASA, «April 1961 – First Human Entered Space» (<https://www.nasa.gov/image-article/april-1961-first-human-entered-space/>).

Правило підрахунку: якщо два прогони отримали різні бали, у таблицю 13 іде нижчий, бо модель, яка розв'язує задачу лише через раз, не можна вважати надійною. Самі розбіжності окремо зафіксовано в таблиці 14.

Таблиця 13: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	gemini-3.5-flash-lite
1. Факт	2	2	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	0 (прогони: 2 і 0)	2	2
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	4	11	12

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	gemini-3.5-flash-lite
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	так	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	ні

Обґрунтування оцінок

1. Факт. Усі три моделі в обох прогонах назвали 1961 рік, що збігається з джерелом. Питання виявилось надто відомим, щоб розрізнити моделі: цей факт у навчальних даних трапляється тисячі разів.

2. Формат. Усі шість відповідей пройшли `json.loads()` без правок, `markdown-обгортку` не додала жодна модель. Проте вміст полів нестабільний. `llama3.2:3b` в одному прогоні написала неточне «введення людини в космос», у другому перейшла на російську й повернула `year` рядком "1961" замість числа; `qwen3:4b` видала поля один раз російською, другий раз англійською; `Gemini` обидва рази відповіла однаково англійською. Мову полів у промпті не задано, тож за рубрикою це не помилка, але програма, яка очікує число в полі `year`, на відповіді `llama` впала б.

3. Стиснення. `llama3.2:3b` в обох прогонах просто скопіювала перші речення оригіналу й загубила 7 із 9 чисел, а в другому прогоні ще й додала вступ «Речі з³веденими текстами...» з китайським ієрогліфом і третє речення. `qwen3:4b` і `Gemini` зберегли всі 9 чисел у двох реченнях. У `qwen` є граматичні похибки («масою 62 мас Сонця», «ставлячись першим доказом»), а слово «близько» біля частини чисел зникло, проте самі числа не спотворено.

4. Код. Функції перевірено на трьох прикладах. Перша функція `llama3.2:3b` дає правильний результат ($\text{distance_km} / 299792458 \times 1000$), хоча коментар «перевод м/с в с» беззмистовний; друга множить на 3600 і завищує час у 3,6 раза (для 299 792,458 км повертає 3,6 с замість 1 с). `qwen3:4b` обидва рази написала правильний чистий код. `Gemini` написала правильну функцію, але обидва рази обгорнула її в `markdown-блок` ````python`, попри вимогу «Лише код»; оскільки обгортка мінімальна, а сам код працює, поставлено 2 бали.

5. Логіка. `qwen3:4b` і `Gemini` в обох прогонах дійшли до 333 повних діб і розписали хід розв'язання. `llama3.2:3b` провалила обидва прогони: у першому через набір беззмистовних обчислень отримала «65 рік», у другому правильно знайшла 8000 годин, але потім помножила їх на 24 замість поділити (докладніше в кроці 7).

6. Українська. llama3.2:3b – 0 балів: у тексті англійські («themselves», «on contrary») та чеські («jej», «hmoti») слова, «астрономічних» латиницею, а також фактичні помилки (нібито планети світять через «горіння» ядерної енергії). qwen3:4b – 1 бал: суть пояснено правильно, але є калька «ядерна фузія» замість «термоядерний синтез», конструкція «відрізняються від планет в тому», помилка узгодження «від зорі, таких як Сонце» і фактична помилка «світло зірки, яка обертається навколо неї». Gemini – 2 бали: природна мова, коректна термінологія, два змістовні абзаци; дрібний недолік – непослідовне вживання «зоря» і «зірка».

Крок 6. Відповіді на питання

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше набрала хмарна gemini-3.5-flash-lite (12/12), далі qwen3:4b (11/12) і llama3.2:3b (4/12). На факті й форматі всі три моделі зрівнялися, а розвели їх стиснення тексту, код, логіка та українська мова; qwen поступилася Gemini лише якістю мови.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Оцінки розійшлися лише в llama3.2:3b на завданні 4: спершу правильна функція, потім множення на 3600. Отже, навіть правильна відповідь малої моделі може бути випадковою, і без тестів такому коду довіряти не можна; qwen3:4b і Gemini були стабільними, а Gemini видавала майже дослівно однакові тексти.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так, у завданні 2 усі три моделі повернули чистий JSON без markdown-обгортки. Але звичка нікуди не зникла: у завданні 4 Gemini обидва рази обгорнула код у ```python, попри «Лише код», тобто на одному промпті модель інструкцію виконала, а на сусідньому – ні.

4. Чи були проблеми з українською мовою – русизми, кальки, перехід на іншу мову?

Так. llama3.2:3b змішувала українську з англійською, чеською й російською і навіть вставила китайський ієрогліф; qwen3:4b писала українською, але з

кальками й граматичними похибками, а поля JSON видала російською та англійською; у Gemini помітних мовних проблем не було.

Крок 7. Приклад провалу

Найпоказовіший випадок – другий прогін завдання 5 у llama3.2:3b: модель правильно почала розв'язання, але на останньому кроці перетворила години на доби не в той бік.

Запит: «Космічний зонд летить зі швидкістю 60000 км/год. Відстань до цілі 480 млн км. Скільки повних діб триватиме політ? Покажи хід розв'язання.»

Повна відповідь моделі (без скорочень і виправлень):

Вираз "повна діб" означає 24 години. Якщо космічний зонд летить з швидкістю 60000 км/год, то він проходить 24 години в час.

Перимітер для обчислення часу, який зонд проїхав:

$$t = d / v$$

$$t = 480000000 \text{ км} / 60000 \text{ км/год}$$

$$t = 8000 \text{ годов}$$

Повні діб триватиме: 8000 годов * 24 години/добу = 192000 доби

Правильна відповідь: 480 000 000 км / 60 000 км/год = 8000 год; 8000 год / 24 год/добу $\approx$ 333,3 доби, тобто 333 повні доби (і ще 8 годин).

Чому так сталося: модель правильно застосувала формулу $t = d / v$ і отримала 8000 годин, бо цей шаблон часто трапляється в навчальних текстах. Але для переведення годин у доби вона відтворила схожий на правильний рядок «N × 24», не перевіряючи напряду перетворення: 192 000 діб – це понад 500 років, що суперечить самим 8000 годин, проте інструктивна модель без етапу міркування не перевіряє власні проміжні результати. Відповідь виглядає як охайний розв'язок із формулою й одиницями виміру, і саме тому помилка небезпечна: правдоподібність міркування і його правильність – різні речі. qwen3:4b і Gemini записали проміжний результат і правильно поділили його на 24.

Крок 8. Відповіді на питання до чат-бота

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Використовують стандартні бенчмарки з автоматичною перевіркою (MMLU для знань, GSM8K для математики, HumanEval для коду з юніт-тестами) та

метрики точного збігу або схожості тексту. Для відкритих відповідей застосовують оцінювання іншою, сильнішою моделлю (LLM-as-a-judge) і попарні порівняння людьми з рейтингом Elo, як у Chatbot Arena, а в продуктах – A/B-тести на реальних користувачах.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

У навчальних даних JSON і код майже завжди стоять у markdown-блоках, а під час донавчання для чат-інтерфейсів модель заохочували за гарно оформлені відповіді. Кілька слів інструкції конкурують із мільйонами прикладів цієї звички, тому на практиці формат забезпечують не промптом, а режимом structured output, схемами й повторним запитом при помилці валідації.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Української в навчальних даних на порядки менше, ніж англійської, а російської – більше, ніж української, тож модель легко «зісковзує» в близьку мову. Токенізатор ділить українські слова на більше дрібних частин, а мала модель має обмежену «місткість», і рідкісні мови в ній представлені найслабше.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

При $temperature > 0$ кожен наступний токен обирається випадково відповідно до ймовірностей, і один інакший токен на початку змінює всю подальшу траєкторію відповіді. Так Llama3.2:3b один раз написала правильну функцію, а наступного разу – з множенням на 3600; додатково на результат впливають особливості паралельних обчислень.

5. Що таке галюцинація мовної моделі і чому модель радше вигадас факт, ніж визнає незнання?

Галюцинація – це впевнена, гладко сформульована, але вигадана відповідь. Модель навчена продовжувати текст найправдоподібнішим чином і не має окремого механізму «я цього не знаю», а під час навчання відповідь заохочувалася частіше, ніж відмова; тому проти галюцинацій використовують RAG, посилання на джерела й явний дозвіл відповідати «не знаю».

Висновок за завданням 5

Оцінка за рубрикою виявилася надійною там, де критерій об'єктивний: `json.loads()`, перевірка чисел у переказі, запуск функції на прикладах і правильна відповідь 333 займають секунди й не залежать від того, хто перевіряє. Найменш надійна частина – завдання 6: межа між 1 і 2 балами за мову суб'єктивна, а два прогони – мінімум, якого ледь вистачило, щоб помітити нестабільність llama на коді. Найпоказовішими виявилися стиснення, логіка й українська мова: саме вони розвели моделі від 4 до 12 балів, тоді як факт і JSON виявилися для всіх надто простими. Для роботи з українськомовним текстом я обрала б хмарну модель, якщо дані не конфіденційні; серед локальних – qwen3:4b, але з обов'язковою вчиткою тексту, а llama3.2:3b для української непридатна.

Завдання 6. Керування випадковістю: temperature і top_p

Експерименти виконано на локальній моделі llama3.2:3b. Варіант 9: творче завдання – назва рекомендаційного сервісу фільмів, фактичне питання – рік випуску першого iPhone. До обох промптів додано «Відповідай лише назвою» та «Відповідай лише роком»: інакше кожна відповідь відрізнялася б супровідним поясненням, і підрахунок унікальних відповідей втратив би сенс. Правильна відповідь – 2007 рік (Apple представила iPhone 9 січня 2007 р.; джерело: Apple Newsroom, <https://www.apple.com/newsroom/2007/01/09Apple-Reinvents-the-Phone-with-iPhone/>).

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	5
temperature	0.7	5	–
temperature	1.5	5	5
top_p (temperature = 0.7)	0.1	1	–
top_p (temperature = 0.7)	0.5	3	–

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
top_p (temperature = 0.7)	1.0	5	–

Відповіді моделі на творче завдання

Налаштування	Відповіді (5 запусків)
temperature = 0.0	Фільмоперсона ×5
temperature = 0.7	«Відомісти»; Фільмовий рекомендаційний сервіс – «Мовчазний Рекомендатор»; Фільмний Афіш; kinopartner; Фільмове кіно-експерт
temperature = 1.5	Форматів та стІLE; «Функаш фільмów»; FilmFix.; Прагафільлері; Goldumovies.Advisor.
top_p = 0.1	Фільмоперсона ×5
top_p = 0.5	Фільмоперсона ×3; Фільмамайстер; Фільмопар
top_p = 1.0	Фільмови на запитання; Фільмамайн; «Фільм на мові твоєї Preferences»; «KINOПРЕДСТАВЛЯЙТЕ»; Фільмаман

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1 («2007.»)
Усі відповіді однакові (так / ні)	так

На фактичному питанні модель не помилилася навіть при temperature = 1.5: усі п'ять відповідей містили 2007 рік. Ймовірність цього факту настільки висока, що навіть «сплющений» розподіл її не перебив. Проте випадковість проявилася в «хвості»: відповіді стали різними за формою («2007 рік.», «2007 р.», «2007.»), а одна містила китайський ієрогліф – «2007年.». Для програми, яка чекає рівно чотири цифри, це вже збій.

Крок 7. Відповіді на питання

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

temperature = 0: агентові потрібна передбачувана однакова відповідь, і мій експеримент дав 10 однакових відповідей із 10. Випадковість тут лише додає шанс зламати структуру, як «2007年» при високій температурі.

2. Яке – генератору варіантів заголовків?

Близько 0,7–1,0: при 0,7 модель дала 5 різних і переважно осмислених назв, при 0 – ту саму «Фільмоперсону» п'ять разів, а при 1,5 половина назв перетворилася на мішанину алфавітів.

3. Чим вплив top_r відрізняється від впливу temperature за характером, а не за силою?

Temperature змінює форму всього розподілу, тож при 1,5 шанс отримали навіть дуже малоймовірні токени – звідси «стILE» і «Goldumovies». top_r нічого не перерозподіляє, а відрізає хвіст: при 0,1 лишився один кандидат «Фільмоперсона», при 0,5 – варіації того самого кореня («Фільмамайстер», «Фільмопар»), тобто різноманітність у межах правдоподібного.

4. Чи означає temperature = 0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

У моєму експерименті всі 10 відповідей збіглися («2007.»), але повної гарантії temperature = 0 не дає: вона прибирає випадковість вибору токена, а не особливості обчислень із плаваючою комою й паралелізації. Для хмарних API розбіжності ще ймовірніші, бо запити обробляються в різних пакетах і на різних серверах.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature = 1.5, і поясніть, чому вона взагалі могла з'явитися.

«Goldumovies.Advisor.» – кирилиця, латиниця й крапки впереміш. При temperature = 1.5 розподіл сплющується, і після токена «Гол» латинські частини слів, які мали мізерну ймовірність, стають конкурентними; кожен наступний токен уже спирається на дивний префікс, тож помилка накопичується.

Крок 8. Відповіді на питання до чат-бота

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Модель видає для кожного токена оцінку (logit) z , і перед softmax ці оцінки діляться на T : $p = \exp(z / T) / \sum \exp(z / T)$. При $T < 1$ різниця між токенами посилюється і розподіл загострюється, при $T > 1$ – згладжується, а при $T \rightarrow 0$ завжди обирається найімовірніший токен.

2. Чим top_p відрізняється від temperature – обидва впливають на різноманітність, але за яким механізмом?

top_p (nucleus sampling) сортує токени за ймовірністю, залишає найменший набір, сумарна ймовірність якого досягає p , перенормовує їх і обирає лише серед них. Temperature змінює співвідношення ймовірностей усіх токенів, а top_p співвідношень не змінює, лише відкидає малоімовірний хвіст.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Зазвичай радять змінювати один параметр, бо вони взаємодіють: висока температура розширює «ядро» top_p, і результат важко передбачити. Комбінацію використовують свідомо, наприклад помірно високу температуру з top_p $\approx 0,9$, щоб отримати різноманітність без сміттєвих токенів.

4. Чому навіть при temperature = 0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Операції з плаваючою комою не асоціативні, а на GPU порядок паралельних додавань може відрізнитися, тож logits двох майже рівних токенів іноді міняються місцями. На сервері додаються динамічне об'єднання запитів у пакети, різні апаратні вузли й оновлення моделі.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для витягу даних, класифікації, JSON і коду – 0–0,2, часто разом зі structured output; для звичайного діалогу – 0,5–0,8; для творчих текстів і генерації ідей – 0,8–1,2, рідко вище, бо далі з'являється сміття, як у моєму експерименті при 1,5.

Висновок за завданням 6

Для наступних лабораторних, де модель має повертати структуровані дані, я оберу `temperature = 0` (не більше 0,2) і залишу `top_p = 1`, бо змінювати обидва параметри немає сенсу. Навіть так відповідь треба перевіряти програмно (`json.loads`, `схема`) і повторювати запит при помилці. Налаштування «за замовчуванням» розраховані на живий діалог (в Ollama `temperature = 0,8`, в OpenAI API – 1,0), тобто навмисно додають випадковість: мій експеримент показав, що вона ламає саме формат (рік із ієрогліфом «年»), навіть коли суть відповіді правильна.

Підсумкове порівняння моделей

Конфігурація: MacBook Air (M1, 2020), 8 ГБ пам'яті, Ollama з моделями llama3.2:3b (2,0 ГБ) і qwen3:4b (2,5 ГБ); хмарна модель – Google gemini-3.5-flash-lite через OpenAI-сумісний API. Швидкість генерації наведено без урахування TTFT, тобто після появи першого видимого токена.

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (3B): llama3.2:3b	Локальна більша (4B): qwen3:4b	Хмарна: gemini-3.5-flash-lite
Швидкість генерації	≈ 64–89 симв./с	≈ 45–80 симв./с	≈ 345 симв./с
TTFT	0,12–0,23 с (теплий старт); 2,8 с (холодний)	18–143 с (приховані роздуми перед відповіддю)	26–60 с (мережа й черга на перевантаженому сервері)
Якість, балів /12	4	11	12
Стабільність формату (завдання 2)	JSON валідний 2/2, але мова полів і тип уеар змінюються між прогонами	JSON валідний 2/2, мова полів змінюється (рос./англ.)	JSON валідний 2/2, відповіді ідентичні
Потрібні ресурси	2,0 ГБ диска, працює на ноутбучі з 8 ГБ пам'яті	2,5 ГБ диска, той самий ноутбук, але відповідь – хвилини	Локально нічого: інтернет і API-ключ
Вартість	Безкоштовно	Безкоштовно	Безкоштовний тариф із лімітами (помилки 503/429), далі оплата за токени
Куди йдуть ваші дані	Лишаються на ноутбучі	Лишаються на ноутбучі	На сервери Google
Працює без інтернету	Так	Так	Ні

Критерій	Локальна мала (3B): llama3.2:3b	Локальна більша (4B): qwen3:4b	Хмарна: gemini-3.5-flash-lite
Коли обирати саме її	Прості шаблонні задачі з миттєвою відповіддю й приватністю: чернетки, автодоповнення, класифікація коротких текстів; не для фактів, логіки й української	Конфіденційні дані, де потрібні логіка й точність, а час не критичний: фонові обробка, перевірка розрахунків	Несекретні дані, коли потрібні найкраща якість і українська мова: тексти для людей, пояснення, код; обов'язково з повторами при збоях

Правило вибору. Локальну модель я візьму, попри її слабкість, коли дані не можна передавати назовні (персональні, медичні, комерційні), коли немає стабільного інтернету або коли потрібна миттєва реакція на простій задачі; якщо ж задача вимагає логіки, краще взяти міркувальну qwen3:4b і змиритися з очікуванням. Хмарну модель я візьму, попри залежність від мережі й передачу даних, коли дані не секретні, а важать якість, українська мова й швидкість генерації довгих текстів, але лише з повторами запитів і запасним провайдером, бо в моїх замірах хмара то повертала 503, то відповідала 100 секунд. Завдяки єдиному формату API ці варіанти можна поєднувати: чутливі запити спрямовувати на локальну модель, решту – в хмару.

Загальні висновки

- 1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;
- 2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;
- 3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;
- 4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: llama3.2:3b проігнорувала вимогу «лаконічно», хмарна Gemini обгорнула код у markdown попри «Лише код», а мова й

типи полів JSON змінювалися між прогонами. Надійність досягається програмною валідацією (`json.loads`, схеми, повторні запити), а не формулюванням;

- 5) Мовні моделі є недетермінованими системами: при `temperature = 0,7` llama3.2:3b один раз написала правильну функцію, а вдруге – ні, а TTFT хмари в трьох повторах коливався від 37 до 100 с. Тому будь-яке вимірювання потребує повторів і медіани, а різницю між двома версіями системи треба перевіряти на повторних прогонах, щоб не сплутати її із шумом;
- 6) Якість моделі залежить від типу задачі, а не описується одним числом: на факті й JSON три моделі зрівнялися, а стиснення тексту, логіка й українська мова розвели їх від 4 до 12 балів;
- 7) `Temperature` – регулятор випадковості, а не якості: для структурованих даних потрібне значення 0, для творчих задач – 0,7–1,0, а при 1,5 модель уже генерує мішанину алфавітів.